

UNIDAD 8 · SEMANAS 13–14

Redes: sockets

📅 Semanas 13–14 · fechas por confirmar </> Laboratorio TD7 (resuelto + desafío IA)

~ 2 animaciones

LABORATORIO TD7 · RESUELTO

Redes con sockets: del servidor mínimo a un servidor web con cinco modelos

Un socket se usa como un archivo (`read`, `write`, `close`), pero conecta procesos de máquinas distintas. Este TD junta todo lo anterior: `fork` por cliente, un hilo por conexión y el productor–consumidor del TD6 para atender a muchos clientes a la vez.

EJERCICIOS

- | | | | |
|---|--|---|--|
| 1 | Lo mínimo de un servidor | 4 | Exploración: estados TCP, la cola de |
| 2 | Servidor eco, iterativo y con <code>fork</code> | | <code>listen</code> , los modelos bajo carga y |
| 3 | Servidor web: iterativo, un hilo por conexión y pool | | UDP |

TD7 · 1

● Resuelto · probado

Lo mínimo de un servidor

Enunciado. Un programa que espera una conexión (pruébalo con `telnet` o `nc`) y termina: `socket` (IPv4, orientado a conexión) → `sockaddr_in` con `INADDR_ANY` y el puerto en orden de red (`htons`) → `bind` → `listen` → `accept` → `close`.

Solución

C tds/td7/sock.c

```
/*
 * TD7 - exercise 1: the server side of a TCP connection, step by step.
 * ./sock [port]      (default 7000)  then, from another terminal:
telnet localhost 7000
 * socket → bind → listen → accept → close.
 */
```

```

#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000

int main(int argc, char *argv[])
{
    int port = argc > 1 ? atoi(argv[1]) : PORT;

    /* 1. An IPv4 (AF_INET), connected, reliable byte stream (SOCK_STREAM =
    TCP). */
    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");

    /* Allow restarting the server right away (otherwise bind fails for ~60
    s: TIME_WAIT). */
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
    "setsockopt");

    /* 2. Local address: any interface of this machine, the chosen port
    (network byte order). */
    struct sockaddr_in addr = { 0 };
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons((uint16_t)port);

    /* 3. and 4. Bind the address, start listening (queue of 1 pending
    connection). */
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 1) == -1, "listen");
    printf("listening on port %d\n", port);
    fflush(stdout);

    /* 5. accept blocks until a client connects and returns a NEW descriptor
    for it. */
    int client = accept(s, NULL, NULL);
    exit_if(client == -1, "accept");
    printf("connection accepted, closing\n");
  
```

```
/* 6. Close the connection and the listening socket. */
exit_if(close(client) == -1 || close(s) == -1, "close");
return EXIT_SUCCESS;
}
```

Salida real

```
$ ./sock 37634 & sleep 0.3; nc -z 127.0.0.1 37634 && echo 'client connected';
wait
listening on port 37634
connection accepted, closing
Connection to 127.0.0.1 37634 port [tcp/*] succeeded!
client connected
```

- `accept` devuelve un **descriptor nuevo** para ese cliente; el socket de escucha sigue disponible para aceptar a otros.
- `htons` / `htonl`: la red usa orden *big-endian*; x86 es *little-endian* (lo viste en el TD1 con `od`).
- `SO_REUSEADDR` permite reiniciar el servidor enseguida: sin él, `bind` falla con `EADDRINUSE` mientras las conexiones anteriores están en `TIME_WAIT`.

Animación A8.1 · Ciclo de vida de una conexión →

TD7 · 2

● Resuelto · probado

Servidor eco, iterativo y con `fork`

Enunciado. Un servidor eco devuelve al cliente todo lo que recibe, salvo `quit`, que cierra la conexión y espera otra. Lanza varios clientes a la vez: ¿qué sucede? Modifica el servidor con `fork` para atender a varios.

Solución

C tds/td7/echo_server.c

```
/*
 * TD7 - exercise 2: echo server. Sends back every line it receives; "quit"
 * closes the
 * connection and the server waits for the next client.
 */
```

```

*  ./echo_server [port] [-f]      -f: one child process per client (fork)
*  Without -f the server is ITERATIVE: a second telnet waits until the first
one leaves
*  (its connection stays in the listen queue). With -f each client gets its
own process.
*/
#include <errno.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000
#define LINE 1024

/* Echo line by line until end of file or "quit". Returns when the client is
done. */
static void serve(int c)
{
    char line[LINE];
    size_t len = 0;

    for (;;) {
        ssize_t n = read(c, line + len, 1);          /* byte by byte: simple,
fine for a lab */
        if (n == -1 && errno == EINTR)
            continue;
        if (n <= 0)                                  /* 0: the client
closed; -1: error */
            return;
        if (line[len] == '\n' || len == LINE - 2) {
            len++;
            if (strncmp(line, "quit", 4) == 0)
                return;
            if (write_all(c, line, len) == -1)
                return;
            len = 0;
        } else {
            len++;
        }
    }
}

```

```

    }
}

int main(int argc, char *argv[])
{
    int port = PORT, use_fork = 0;
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-f") == 0) use_fork = 1;
        else port = atoi(argv[i]);
    }

    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
"setsockopt");
    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_port =
htons((uint16_t)port),
                               .sin_addr.s_addr = htonl(INADDR_ANY) };
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 8) == -1, "listen");
    printf("echo server on port %d (%s)\n", port, use_fork ? "one process per
client" : "iterative");
    fflush(stdout);

    /* Children that finish must not stay zombies: ignoring SIGCHLD makes the
kernel reap them. */
    if (use_fork)
        exit_if(signal(SIGCHLD, SIG_IGN) == SIG_ERR, "signal");

    for (;;) {
        int c = accept(s, NULL, NULL);
        if (c == -1 && errno == EINTR)
            continue;
        exit_if(c == -1, "accept");

        if (!use_fork) {
            serve(c);
            exit_if(close(c) == -1, "close");
            continue;
        }
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0) {
            exit_if(close(s) == -1, "close");           /* the child does not

```

```
accept */
    serve(c);
    _exit(EXIT_SUCCESS);
}
exit_if(close(c) == -1, "close");      /* the parent does not
talk to the client */
}
```

Salida real

```
$ python3 tcp_client.py echo 22751 'hola\nmundo\nquit'
sent: hola      received: hola
sent: mundo     received: mundo
```

Salida real

```
$ python3 tcp_client.py blocked 22751      # iterative server
client B got nothing after 1 s: the server is busy with client A
```

Salida real

```
$ python3 tcp_client.py blocked 34885      # server with fork
client B got 'hi\n' after 0.00 s
```

El servidor iterativo atiende a un cliente por vez: mientras el cliente A está conectado, B queda en la cola de `listen` y no recibe respuesta (la prueba lo mide: nada en 1 s). Con `-f`, cada conexión tiene su proceso y B recibe el eco al instante. El padre cierra el socket del cliente y el hijo el de escucha; `SIGCHLD` ignorada evita los zombis. La versión heredada detectaba `quit` con `strstr` sobre un búfer sin terminador; ahora se lee por líneas.

TD7 · 3

● Resuelto · probado

Servidor web: iterativo, un hilo por conexión y pool de 8 hilos

Enunciado. El cliente (un navegador) envía `GET archivo`; el servidor responde `HTTP/1.0 200 OK`, el tipo MIME y el contenido. Usa `stat` para saber si existe y su tamaño, `mmap` para mapearlo y `write` para enviarlo. Después: (1) un hilo nuevo por conexión; (2) 8

hilos creados al inicio que esperan conexiones — el servidor *produce* conexiones y los hilos las *consumen*.

Solución

C tds/td7/http_server.c

```
/* Handles one request on the connected socket c. Never exits the process. */
static void handle(int c)
{
    char req[REQUEST], path[1024], full[2048];
    ssize_t n = read(c, req, sizeof req - 1);
    if (n ≤ 0)
        return;
    req[n] = '\0';

    if (sscanf(req, "GET %1023s", path) ≠ 1) {
        reply_error(c, "400 Bad Request");
        return;
    }
    if (strstr(path, "..") ≠ NULL) {                                /* no escape from root */
        /*
        reply_error(c, "403 Forbidden");
        return;
        */
    }
    if (strcmp(path, "/") = 0)
        strcpy(path, "/index.html");
    snprintf(full, sizeof full, "%s%s", root, path);

    struct stat st;
    if (stat(full, &st) = -1 || !S_ISREG(st.st_mode)) {
        reply_error(c, "404 Not Found");                            /* PG204 answered
        "201" by mistake */
        return;
    }
    int fd = open(full, O_RDONLY);
    if (fd = -1) {
        reply_error(c, "403 Forbidden");
        return;
    }
    char head[256];
    int hn = snprintf(head, sizeof head,
                      "HTTP/1.0 200 OK\r\nContent-Type: %s\r\nContent-Length:
                      %lld\r\n\r\n",
```

```

        mime_type(full), (long long)st.st_size);
    if (write_all(c, head, (size_t)hn) == -1) {
        close(fd);
        return;
    }
    if (st.st_size > 0) {
        /* mmap of 0 bytes is
an error */
        void *data = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE,
fd, 0);
        if (data != MAP_FAILED) {
            (void)write_all(c, data, (size_t)st.st_size); /* loops over
partial writes */
            munmap(data, (size_t)st.st_size);
        }
    }
    close(fd);
}

/* ----- thread per connection ----- */
static void *connection_thread(void *arg)
{
    int c = (int)(intptr_t)arg;
    handle(c);
    close(c);
    return NULL;
}

/* ----- pool: bounded queue of connected sockets ----- */
static int queue[QUEUE];
static int q_in = 0, q_out = 0;
static sem_t q_free, q_used;
static pthread_mutex_t q_lock = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *arg)
{
    (void)arg;
    for (;;) {
        sem_wait(&q_used);
        pthread_mutex_lock(&q_lock);
        int c = queue[q_out];
        q_out = (q_out + 1) % QUEUE;
        pthread_mutex_unlock(&q_lock);
        sem_post(&q_free);
        handle(c);
        close(c);
        /* wait for a connection */
    }
}

```

```
}
return NULL;
}
```

Salida real

```
$ curl -s -i http://localhost:PORT/ | tr -d '\r' # mode pool
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 20

<h1>FS0 · UTI</h1>
```

Salida real

```
$ curl -s -o /dev/null -w '%{http_code}' ../missing.html ; curl --path-as-is
.../../etc/passwd
404
403
```

MODO	CÓMO ATIENDE	LÍMITE
<code>-m iterative</code>	Un cliente por vez	Un cliente lento bloquea a todos
<code>-m fork</code>	Un proceso hijo por conexión	Crear un proceso cuesta varias veces más que un hilo (medido en el TD6 · 5)
<code>-m thread</code>	Un hilo nuevo por conexión (<code>pthread_detach</code>)	Miles de conexiones = miles de hilos (memoria, cambios de contexto)
<code>-m pool</code>	8 hilos fijos + cola acotada (semáforos + mutex)	Recursos acotados; 8 clientes lentos lo saturan (TD7 · 4)
<code>-m epoll</code>	Un hilo y un bucle de eventos: solo atiende sockets que ya tienen datos	Esta versión simple aún escribe bloqueando; nginx o Node.js también escriben sin bloquear

Las pruebas lanzan 40 peticiones concurrentes (8 a la vez) en cada modo y comprueban 40 respuestas 200 y un binario idéntico al original. Seguridad mínima:

una ruta con `..` recibe **403** (el cliente no puede salir de la carpeta servida) y lo que no existe, **404**. El servidor ignora SIGPIPE para no morir si un cliente se desconecta a mitad de la respuesta.



Corregido y completado

La versión heredada respondía `201 Not found` (el código correcto es `404`), no evitaba `..`, no enviaba `Content-Length`, ignoraba las escrituras parciales, no abría el archivo con comprobación de error y no incluía las versiones con hilos ni con pool que pide el enunciado.

[Animación A8.2 · Modelos de servidor →](#)

TD7 · 4

● Nuevo · medido en el equipo del curso

Exploración: lo que el núcleo hace con tus sockets

Enunciado. (1) Lee con `getsockopt(TCP_INFO)` el estado de cada socket en cada paso de una conexión. (2) Averigua qué limita el `backlog` de `listen`. (3) Mide los cinco modos del servidor web con clientes lentos. (4) Compara TCP y UDP. (5) Comprueba el error de `htons` de las diapositivas originales y resuelve nombres con `getaddrinfo`. (6) Crea un socket local.

1 · Estados TCP (`tcp_states.c`)



C tds/td7/tcp_states.c

```

    struct tcp_info ti;
    socklen_t len = sizeof ti;
    exit_if(getsockopt(s, IPPROTO_TCP, TCP_INFO, &ti, &len) == -1,
"getsockopt");
    return ti;
}

static const char *state(int s)
{
    if (s == -1)
        return "-";
    unsigned st = info(s).tcpi_state;
    return st < sizeof NAMES / sizeof NAMES[0] ? NAMES[st] : "?";
}

/* One line per step: the state of each socket. For a listening socket the
kernel reports the
* accept queue in tcpi_unacked (connections waiting for accept) and
tcpi_sacked (backlog). */
static void show(const char *step, int l, int c, int a)
{
    struct tcp_info li = info(l);
    printf("%-26s | listener %s (queue %u/%u) | client %-11s | server %s\n",
        step, state(l), li.tcpi_unacked, li.tcpi_sacked, state(c),
state(a));
}

```

Salida real

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -
| server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED
| server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED
| server ESTABLISHED
write/read                  | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2
| server CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED
| server CLOSED
                           | the client's end moved to a TIME_WAIT entry of
the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0         0      127.0.0.1:42772   127.0.0.1:PORT
```

Tres hechos que la salida muestra: la conexión está **establecida antes de** `accept` (el saludo lo hace el núcleo; la cola pasa a 1/4); el que cierra primero pasa por `FIN_WAIT2` mientras el otro queda en `CLOSE_WAIT` hasta que su `read` devuelve 0; y tras el cierre el núcleo conserva una entrada **`TIME_WAIT`** (60 s) que `ss` muestra aunque el programa ya terminó. Por eso los servidores usan `SO_REUSEADDR`.

2 · Qué limita el backlog (`backlog.c`)

Salida real

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5,
SYN_SENT 3
accept 5 (total 5), wait:   accept queue 3 | clients ESTABLISHED 8,
SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

Con `listen(fd, 4)` Linux completa **`backlog + 1 = 5`** saludos y descarta el SYN de los demás, que siguen en `SYN_SENT` y lo **reenvían** (1 s, luego 2 s, 4 s...). Cuando el servidor acepta, la cola se vacía y los reenvíos entran. El backlog no limita cuántos clientes atiende el servidor: limita cuántas conexiones terminadas pueden esperar a `accept`.

💡 Para curiosos: SYN cookies

Salida real

```
$ nstat -n; ./backlog 2 8; nstat | grep -E
'Syncookies|ListenOverflows'
listen(2), 8 clients connect
0.3 s, no accept yet:          accept queue 3 | clients
ESTABLISHED 3, SYN_SENT 5
accept 3 (total 3), wait:      accept queue 3 | clients
ESTABLISHED 8, SYN_SENT 0
accept 3 (total 6), wait:      accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
accept 0 (total 6), wait:      accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
TcpExtSyncookiesSent 2
TcpExtListenOverflows 7
```

Con `listen(fd, 2)` pasa algo más fino: cuando la cola de saludos a medio hacer también se llena, el núcleo responde con **SYN cookies** (sin guardar estado, una defensa contra la inundación de SYN). Si al llegar el ACK final la cola de accept está llena, ese ACK se pierde: el cliente cree estar ESTABLISHED, pero el servidor no sabe de él. Los contadores del núcleo (`nstat`) lo confirman. Por eso el backlog de un servidor real es grande (en este equipo, `net.core.somaxconn` = 4096).

3 · Los cinco modelos bajo carga (`load.c`)



32 clientes: primero 1 (y luego 8) que tardan 1 s en enviar su petición, como un cliente en una red mala o un ataque *slowloris*; 20 ms después, los rápidos. Se mide la latencia de cada grupo, de `connect` a la respuesta completa.

Salida real

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 982 ms, max 983 ms
```

Salida real

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 981 ms, max 981 ms
```

Salida real

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 1 ms, max 2 ms
```

Iterativo: un solo cliente lento hace esperar ~1 s a todos. Pool de 8: aguanta 1 lento, pero con 8 los ocho hilos quedan bloqueados en `read` y vuelve a pasar lo mismo. `fork`, hilo por cliente y `epoll` responden a los rápidos en pocos milisegundos (las salidas de `fork` y `thread` están en `tests/out/td7/`). El bucle de eventos lo logra con **un solo hilo**:

C tds/td7/http_server.c

```

/* ----- event loop: one thread watches every socket ----- */
static void event_loop(int s)
{
    int ep = epoll_create1(0);
    exit_if(ep == -1, "epoll_create1");
    struct epoll_event ev = { .events = EPOLLIN, .data.fd = s }, ready[64];
    exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, s, &ev) == -1, "epoll_ctl");
    for (;;) {
        int n = epoll_wait(ep, ready, 64, -1); /* sleeps until some socket
is readable */
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "epoll_wait");
        for (int i = 0; i < n; i++) {
            int fd = ready[i].data.fd;
            if (fd == s) { /* a new connection: watch
it too */
                int c = accept(s, NULL, NULL);
                if (c == -1)
                    continue;
                ev.data.fd = c;
                exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, c, &ev) == -1,
"epoll_ctl");
            } else { /* the request has arrived:
read won't wait */
                exit_if(epoll_ctl(ep, EPOLL_CTL_DEL, fd, NULL) == -1,
"epoll_ctl");
                handle(fd);
                close(fd);
            }
        }
    }
}

```

Animación A8.2 · Modelos de servidor →

4 · TCP frente a UDP (udp.c)



Salida real

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3
"two", recvfrom #3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8
received, 9992 lost
kernel counter Udp RcvbufErrors grew by 9992
```

TCP es un **flujo de bytes**: tres `write` llegan en un solo `read` (igual que en una tubería), así que el protocolo de la aplicación debe marcar dónde termina cada mensaje. UDP conserva los **límites** de cada datagrama, pero no espera al receptor: con el búfer lleno, el núcleo descarta los datagramas sin avisar a nadie, incluso dentro de la misma máquina, y solo lo anota en `/proc/net/snmp`.

5 · Direcciones y puertos (`addr.c`) · 6 · Socket local (`local.c`)

Salida real

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000: the kernel bound port 22555
sin_port = htons(7000): the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80): bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110,
sockaddr_storage 128
```

Salida real

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path
holds 108 bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

El ejemplo de la diapositiva 187 asignaba `sin_port = 7000` sin `htons`: en un procesador *little-endian* los dos bytes se leen al revés y el núcleo abre el puerto **22555**. `gethostbyname` (también en esa diapositiva) solo sabe de IPv4, no es seguro con hilos y salió de POSIX en 2008: `getaddrinfo` resuelve IPv4 e IPv6 con la misma llamada. Un socket `AF_UNIX` no usa 127.0.0.1 (eso es `AF_INET` por la interfaz de bucle local): su dirección es un **archivo de tipo socket**, y el núcleo le dice al servidor quién se conectó (`SO_PEERCRED`), algo que systemd, D-Bus o Docker usan para decidir permisos.

Animación A8.1 · Ciclo de vida de una conexión →

Lista de cotejo

Desafío con IA generativa · Un servidor `epoll` que nunca se bloquea



Objetivo. Convertir el modo `-m epoll` de `http_server.c` (que todavía lee y escribe bloqueando) en un bucle de eventos completo, con sockets no bloqueantes y escrituras parciales, y medirlo frente al modo de un hilo por cliente con miles de conexiones.

Prompt inicial. Úsalo en la herramienta que prefieras (Claude, ChatGPT, Gemini, Copilot, Codex...) con la versión *resuelta* del TD como contexto. Pide primero un plan y revisa cada paso antes de aceptarlo.

Prompt

Tengo un servidor HTTP en C con cinco modos; el modo `epoll` usa un solo hilo, pero `handle()` hace un `read` y un `write` que pueden bloquear [pega `http_server.c` y `load.c`]. Ayúdame a: (1) usar `accept4(..., SOCK_NONBLOCK)` y guardar por conexión un estado (leyendo la petición hasta `\r\n\r\n`, escribiendo la respuesta con un desplazamiento), registrando `EPOLLOUT` solo cuando `write` devuelve `EAGAIN`; (2) extender `load.c` para abrir 2000 conexiones que envían muy despacio; (3) comparar el modo `epoll` con `-m thread` midiendo latencia de los clientes rápidos, número de hilos (`ps -o nlwp`) y memoria (`ps -o rss`). Explicame el problema C10K y qué recurso del SO limita a cada modo.

Cómo verificar el resultado

- Un cliente que lee muy despacio un archivo grande (`curl --limit-rate 10k`) no retrasa a los demás: `./load` lo muestra.
- `tests/run.sh td7` sigue pasando y el archivo binario llega idéntico (`cmp`).
- La tabla sale de tus mediciones y dice en qué equipo se hicieron.
- `ulimit -n` explica al menos un fallo que observes al subir la concurrencia.

Declara el uso de IA: herramienta, prompts relevantes, qué verificaste tú y qué corregiste. Lo que cuenta es que puedas explicar y comprobar cada decisión.